

sentiment_classifier

December 7, 2025

1 Introductie

Deze week ga je een sentiment classifier toevoegen aan je Video Informatie Systeem (VIS). De sentiment classifier produceert een label (positief of negatief sentiment) voor een video en dit kun je in de database opslaan. Om dit te doen heb je eerst kennis nodig over machine learning.

Machine learning is een manier waarop computers zelf kunnen leren van voorbeelden in plaats van dat een programmeur alle regels moet voorschrijven. Bij classificatie leert een computer bijvoorbeeld om dingen in hokjes te plaatsen, zoals het verschil zien tussen katten- en hondenfoto's. In Natural Language Processing (NLP) gaat het om het begrijpen van menselijke taal, zodat een computer kan werken met tekst of spraak. Dat kan bijvoorbeeld handig zijn bij het automatisch herkennen of een bericht positief of negatief klinkt. Zo maakt machine learning slimme toepassingen mogelijk die ons dagelijks leven makkelijker maken.

1.1 Importeer de nodige packages

Voor het implementen van een sentiment classifier met NLP technieken hebben we veel import nodig. Per import wordt uitgelegd waarvoor deze dient.

```
[1]: # Numpy is standaard nodig voor alle andere functies
import numpy as np

# We laden straks movie reviews in ons geheugen. Dit zijn losse tekst bestanden
↳ (50.000 stuks).
# Sklearn heeft een speciale functie die dit voor je doet.
from sklearn.datasets import load_files

# Voordat we tekst kunnen gebruiken voor classificatie moeten we het
↳ transformeren naar een Bag of Words model (vectoren).
# Dit doen we met de countVectorizer.
from sklearn.feature_extraction.text import CountVectorizer

# Om te valideren of ons model goed werkt moeten we het testen op testdata. Dit
↳ doen we door de dataset
# op te splitsen in een trainset en een testset.
from sklearn.model_selection import train_test_split
```

```
# In deze oefening gaan we meerdere Machine learning modellen met elkaar
↪vergelijken.
# We evalueren er vier in totaal
from sklearn.naive_bayes import CategoricalNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn import svm

# Voor het evalueren van een classificatie model op de testset gebruiken we de
↪accuracy en een confusion matrix.
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix

# We visualeren de confusion matrix met de volgende twee imports
import matplotlib.pyplot as plt
import seaborn as sns
```

2 De Machine Learning Pipeline

Als je een machine learning analyse doet zal dit altijd gebeuren voor een vast aantal stappen. Dit noemen we ook wel de machine learning pipeline. Hier volgen de stappen.

Data verzamelen

Dit is de eerste stap, waarin je alle benodigde data verzamelt.

Data kan komen uit bestanden (CSV, Excel), databases, sensoren of online bronnen.

Voor NLP kan dit bijvoorbeeld een verzameling e-mails, tweets of reviews zijn.

Hiermee leggen we ook meteen de grote zwakte bloot van machine learning: zonder data of slechte data -> geen machine learning.

Data schoonmaken en voorbereiden (preprocessing)

Voordat we de data kunnen gebruiken moeten we deze eerst voorbereiden. Ruwe data is meestal nooit meteen bruikbaar voor een ML model. De volgende bewerkingstappen zijn essentieel.

Verwijderen of invullen van ontbrekende waarden.

Omzetten van tekst naar een formaat dat een computer kan begrijpen (bijvoorbeeld woorden

Normaliseren of schalen van getallen, zodat ze vergelijkbaar zijn.

Opsplitsen van de data in een trainingsset (om het model te leren) en een testset (om te

Feature engineering en selectie

Nieuwe eigenschappen (features) maken die het model kunnen helpen betere voorspellingen te doen.

Onbelangrijke of overbodige eigenschappen verwijderen om het model eenvoudiger en sneller te maken.

In NLP kan dit bijvoorbeeld betekenen dat je woorden telt, woordfrequenties berekent of speciale tekstkenmerken gebruikt.

Modelkeuze en training

Kiezen welk type machine learning model het beste past bij het probleem.

Voor classificatie: bijvoorbeeld Decision Tree, Random Forest, of Logistic Regression.

Voor NLP: bijvoorbeeld Naive Bayes, LSTM of Transformer-modellen.

```
</li>  
<li>Het model trainen op de trainingsdata, zodat het patronen leert herkennen.</li>  
</ul>
```

Model testen en Evaluatie van het model

Het model testen op de testset die het nog niet heeft gezien.

Prestaties meten met geschikte metrics, zoals:

Accuracy (hoe vaak het model het goed heeft)

Precision/Recall (bijvoorbeeld voor spamdetectie)

F1-score (combinatie van precisie en recall)

Confusion Matrix

```
</li>  
</ul>
```

Deployen en nieuwe voorspellingen maken

Het getrainde model gebruiken om nieuwe data te voorspellen of classificeren.

In NLP kan dit bijvoorbeeld betekenen dat je automatisch e-mails als spam of geen spam labelt, of sentiment analyseert in reviews.

Het model beschikbaar maken via een webapplicatie, API, Ron Server, of ander systeem zodat anderen het kunnen gebruiken.

3 1 Data verzamelen

Om een sentiment classifier te bouwen hebben we voorbeelden nodig van teksten met een positief sentiment en voorbeelden van een negatief sentiment. Laten we naar een tweetal voorbeelden kijken. Dit zijn reviews van de IMDB website. Als eerste een positief voorbeeld:

I'm a male, not given to women's movies, but this is really a well done special story. I have no personal love for Jane Fonda as a person but she does one Hell of a fine job, while DeNiro is his usual superb self. Everything is so well done: acting, directing, visuals, settings, photography, casting. If you can enjoy a story of real people and real love - this is a winner..

Vraag: vind jij deze review positief? Waar blijkt dit uit? Hoe zou jij een computer leren dat dit een positieve review is?

Nu een voorbeeld van een negatieve review.

Not only is it a disgustingly made low-budget bad-acted movie, but the plot itself is just STUPID!!! A mystic man that eats women? (And by the looks, not virgin ones) Ridiculous!!! If you've got nothing better to do (like sleeping) you should watch this. Yeah right..

Wederom de vraag: vind jij deze review negatief? Waar blijkt dit uit? Hoe zou jij een computer leren dat dit een negatieve review is?

3.0.1 Data downloaden

We gaan nu de IMDB review dataset downloaden. Dit kun je doen via de volgende link: <https://ai.stanford.edu/~amaas/data/sentiment/>

Deze file is gecompriemd en moet je uitpakken. Je ziet dan een folder structuur met 'train' en 'test'. En in die folders heb je weer folders 'pos' en 'neg'.

3.0.2 Beschrijvende analyse

Nu volgen een aantal vragen over deze dataset. Lees hiervoor de README file.

1. Uit hoeveel reviews bestaat deze dataset? Hoeveel daarvan zijn er positief en hoeveel zijn er negatief?
2. hoe wordt er rekening gehouden met Bias in de dataset?
3. Hoe krijgt een review zijn label of het positief of negatief is?

3.0.3 data inladen

Nu gaan we de data inladen. Er is een speciale functie in sklearn die de hele mappen structuur herkent en alles netjes in een dataframe laadt. Dat scheelt een hoop werk. Je hoeft alleen het pad naar de folder op te geven.

```
[2]: path_to_imdb = "data/aclImdb/train/"
# Load the IMDB dataset
movie_reviews_data = load_files(path_to_imdb, categories=['neg', 'pos'],
    ↪shuffle=True, random_state=42)
X_train = movie_reviews_data.data
y_train = movie_reviews_data.target
```

4 Stap 2 en 3 Feature engineering

Voordat we tekst kunnen gebruiken in een classificatie model moeten we het eerst converteren naar getallen. Een computer kan namelijk niet rekenen met tekst. We gebruiken hier het Bag of Words (BOW) model. Dit is wat je noemt een language model.

4. Zoek op internet op (of gebruik ChatGPT) wat Bag of Words is en hoe het werkt.

Naast BOW moeten we tekst preprocessen (stap 2), features maken (engineeren) en het aantal features kiezen. Hiervoor heeft de functie CountVectorizer een aantal argumenten. Dit is een alles-in-een functie.

5. Waarvoor dient het argument stopwords? Wat zijn stopwords?
6. waarvoor dient het argument max_features?

7. Waarvoor dienen de argumenten `min_df` en `max_df`?
8. Waarvoor dient het argument `binary`?

Opdracht: de argumenten in de functie zijn nu niet goed afgesteld. En jou de taak om deze zo goed mogelijk in te stellen zodat je een hoge accuracy zult behalen. Een accuracy van >80% is goed haalbaar.

```
[3]: cv = CountVectorizer(stop_words=None, max_features=10000,min_df=20,max_df=0.
      ↪87,binary=True)
```

Een opmerking over de python code. sklearn heeft een structurele manier van hoe alle functies zijn opgebouwd. Zij zijn als volgt:

1. functie declareren (bijvoorbeeld `CountVectorizer()`)
2. het trainen van de functie met `fit`.
3. een getrainde functie gebruiken op nieuwe data met `transform`
4. als het om classificatie gaat: getrainde functie gebruiken met `predict`.

Kortom je ziet altijd functies terug met `fit`, `transform`, of `predict`.

```
[4]: cv.fit(X_train)
      X_train_bow = cv.transform(X_train)
```

```
[5]: len(X_train_bow.toarray())
```

```
[5]: 25000
```

5 4 Model trainen

Nu we een Bag of Words model hebben voor onze tekst (we hebben de tekst pre-processed) kunnen we het stoppen in een classifier. Er zijn heel veel classifiers beschikbaar. We kijken welke het beste presteert op onze data.

Oefening: uncomment telkens een classifier en run de gehele code. Schrijf op welke accuracy je behaalt.

```
[6]: from sklearn.naive_bayes import MultinomialNB
      # keep your CountVectorizer as-is
      clf = MultinomialNB()
      #clf = LogisticRegression(random_state=0)
      #clf = RandomForestClassifier(max_depth=2, random_state=0)
      #clf = MLPClassifier(random_state=1, max_iter=300)
      #clf = svm.SVC()
```

Train de classifier. Zie hoe gebruik gemaakt wordt van de `fit` functie.

```
[7]: clf.fit(X_train_bow.toarray(), y_train)
```

```
[7]: MultinomialNB()
```

6 5 Model testen

Nu het model getraind is gaan we het testen op test data. De train test split is al voor ons gedaan. De dataset is al opgeplits in een train set en een test set.

```
[8]: path_to_imdb = "data/aclImdb/test/"
     # Load the IMDB dataset
     movie_reviews_data = load_files(path_to_imdb, categories=['neg', 'pos'],
     ↪ shuffle=True, random_state=42)
     X_test = movie_reviews_data.data
     y_test = movie_reviews_data.target
```

```
[9]: X_test_cv = cv.transform(X_test)
```

```
[10]: predicted = clf.predict(X_test_cv.toarray())
```

6.0.1 Validatie met nauwkeurigheid

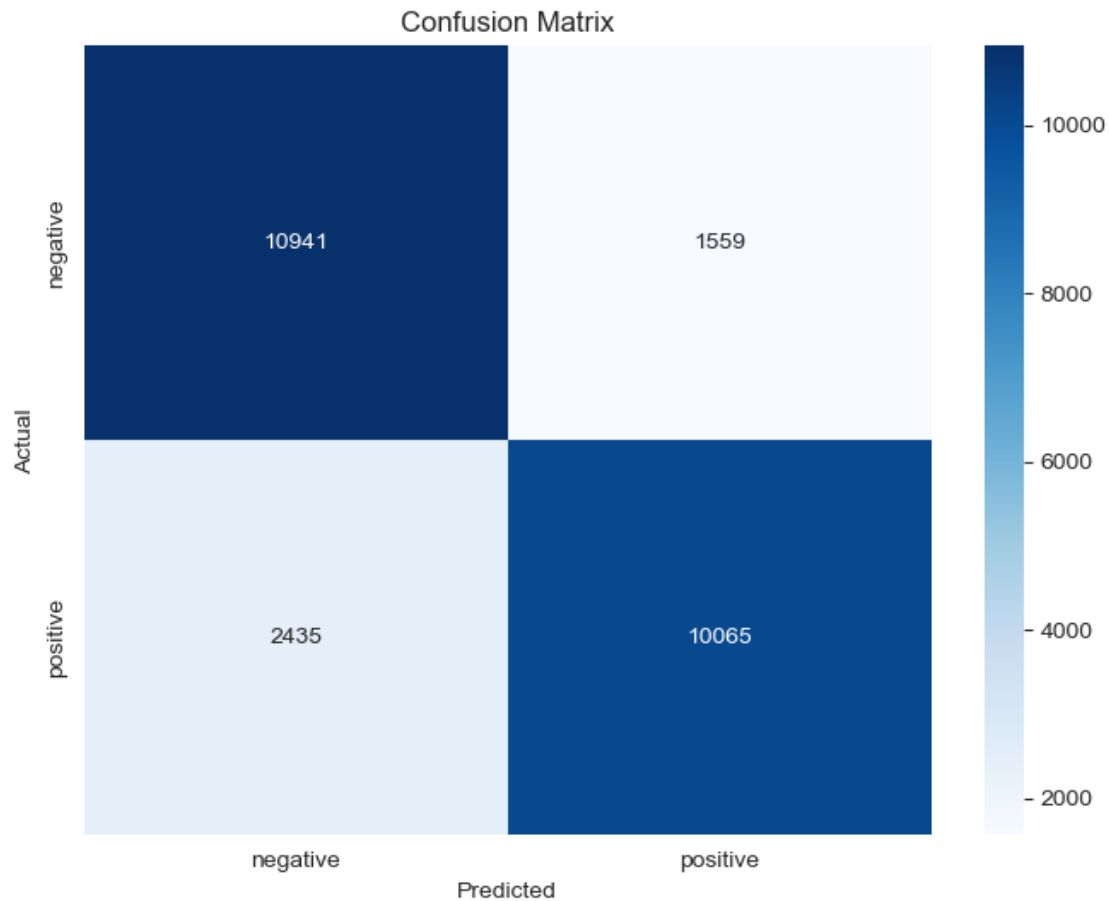
We zijn nu bijna klaar. We hebben voorspellen gedaan op de testset. Nu gaan we de echte labels vergelijken met onze voorspellingen. We bereken hiervoor de accuracy en we maken een confusion matrix.

9. Hoe wordt de accuracy berekend?
10. Wat is een confusion matrix?

```
[11]: acc = accuracy_score(y_test, predicted)
     print(acc)
```

0.84024

```
[12]: # Plot confusion matrix
     cm = confusion_matrix(y_test, predicted)
     plt.figure(figsize=(8, 6))
     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['negative',
     ↪ 'positive'], yticklabels=['negative', 'positive'])
     plt.xlabel('Predicted')
     plt.ylabel('Actual')
     plt.title('Confusion Matrix')
     plt.show()
```



7 Tot slot

We hebben nu een volledige classificatie pipeline doorlopen.

- Is het gelukt om de accuracy boven de 80% te krijgen?
- Begrijp je alle NLP termen en machine learning termen die zijn behandeld in dit document? (bag of words, stopwords, train data/test data, confusion matrix, etc).

```
[13]: import pickle
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("vectorizer", cv),
    ("classifier", clf)
])

pipe.fit(X_train, y_train)
```

```
with open("models/sentiment_pipeline.pkl", "wb") as f:
    pickle.dump(pipe, f)

print(" sentiment_pipeline.pkl getraind en opgeslagen")
```

```
sentiment_pipeline.pkl getraind en opgeslagen
```